CO3 Assignment 3: C Programming Solutions

Course: Problem Solving & Programming in C (Arrays, Strings, Searching & Sorting)

Student A & B: [Names] | **Roll Nos:** [Roll Numbers]

Q1 — Searching a Library Catalogue

1. Completed Code (Binary Search & Linear Search)

```c
#include <stdio.h>

int steps; /* comparisons made by the last search */

int linear_search(const int a[], int n, int key) {
    steps = 0;
    for (int i = 0; i < n; i++) {
        steps++;
        if (a[i] == key) return i;
    }
    return -1;
}

int binary_search(const int a[], int n, int key) {
    steps = 0;
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        steps++;
        if (a[mid] == key) return mid;
        else if (a[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

int main(void) {
    int accession[] = {1021, 1048, 1103, 1190, 1222, 1305, 1377, 1402, 1460, 1533};
    int n = sizeof accession / sizeof accession[0];
    int keys[] = {1021, 1305, 1533, 1400};
    for (int k = 0; k < 4; k++) {
        int pos = binary_search(accession, n, keys[k]);
        printf("Book %d -> index %d, comparisons = %d\n", keys[k], pos, steps);
    }
    return 0;
}
```

Why a Missing Book Needs as Many Comparisons as the Worst Successful Search: Binary search halves the search interval until either `a[mid] == key` or `low > high`. A missing key must descend all the way to a leaf of the binary decision tree ($\lfloor \log_2 n \rfloor + 1 = \lfloor \log_2 10 \rfloor + 1 = 4$ comparisons), matching the depth of the deepest valid element (such as 1533 at index 9).

2. Student A — Pseudocode, Mathematical Derivation & Predictions

```
ALGORITHM BinarySearch(A, n, key)
  low ← 0; high ← n - 1; steps ← 0
  WHILE low ≤ high DO
      mid ← low + (high - low) / 2
      steps ← steps + 1
      IF A[mid] == key THEN RETURN mid
      ELSE IF A[mid] < key THEN low ← mid + 1
      ELSE high ← mid - 1
  END WHILE
  RETURN -1
END ALGORITHM
```

- **Why Sorted Data & Overflow-Safe Midpoint:** Binary search discards half the array by assuming all elements left of `mid` are $\leq A[mid]$ and all right are $\geq A[mid]$; unsorted data violates this invariant. Computing `mid = low + (high - low) / 2` prevents signed integer overflow when `low + high > INT_MAX` (for arrays exceeding 2^{30} elements).
- **Predicted Comparisons for $N = 100{,}000$:** Linear search requires **1** (best), **50,000.5** (average), and **100,000** (worst/missing) comparisons. Binary search requires **1** (best), $\approx \log_2(100{,}000) - 1 \approx 15.6$ (average), and $\lfloor \log_2(100{,}000) \rfloor + 1 = 17$ (worst/missing) comparisons.

3. Student B — Experimental Results Across Array Sizes

Catalogue Size (N)	Linear (First / Mid / Last / Missing)	Binary (First / Mid / Last / Missing)	Agreement with Prediction
`10` (Trial)	1 / 6 / 10 / 10	3 / 3 / 4 / 4	Exact match ($\lfloor \log_2 10 \rfloor + 1 = 4$)
`1,000`	1 / 500 / 1,000 / 1,000	9 / 1 / 10 / 10	Exact match ($\lfloor \log_2 1{,}000 \rfloor + 1 = 10$)
`10,000`	1 / 5,000 / 10,000 / 10,000	13 / 1 / 14 / 14	Exact match ($\lfloor \log_2 10{,}000 \rfloor + 1 = 14$)
`100,000`	1 / 50,000 / 100,000 / 100,000	16 / 1 / 17 / 17	Exact match ($\lfloor \log_2 100{,}000 \rfloor + 1 = 17$)

4. Joint Design Decision & Alternative Complexity Analysis

- **Daily Book Additions Decision:** Inserting into a sorted array costs `O(n)` shifts per insertion. To decide, we need: (1) daily insertion rate I vs. daily kiosk search rate S, and (2) whether updates can be batched overnight. If $S \gg I$ (e.g., 5,000 searches/day vs. 20 new books/day), keeping the array sorted —or buffering new arrivals in a small unsorted overflow list searched linearly alongside binary search on the main array—is optimal.
- **Hash Table vs. Interpolation Search vs. Binary Search:** A hash table with linear probing achieves `O(1)` average lookup time at the cost of `O(n)` extra space (typically 1.3×–2× array size to keep load factor $\alpha \leq 0.7$) and loses sorted order. Interpolation search achieves `O(log log n)` average time with `O(1)` extra space when accession numbers are uniformly distributed, but degrades to `O(n)` if clustered, whereas binary search guarantees `O(log n)` worst-case time with `O(1)` extra space.

Q2 — Sorting the Examination Hall Seating Register

1. Shared Implementations & Student A Tracing / Concepts

```
/* 1. Bubble Sort with Early-Exit Flag */
void bubble_sort(int a[], int n, long *cmp, long *swp) {
    *cmp = *swp = 0;
    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;
        for (int j = 0; j < n - 1 - i; j++) {
            (*cmp)++;
            if (a[j] > a[j + 1]) {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                (*swp)++; swapped = 1;
            }
        }
        if (!swapped) break;
    }
}

/* 2. Selection Sort */
void selection_sort(int a[], int n, long *cmp, long *swp) {
    *cmp = *swp = 0;
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;
        for (int j = i + 1; j < n; j++) {
            (*cmp)++;
            if (a[j] < a[min_idx]) min_idx = j;
        }
        if (min_idx != i) {
            int t = a[i]; a[i] = a[min_idx]; a[min_idx] = t;
            (*swp)++;
        }
    }
}

/* 3. Insertion Sort */
void insertion_sort(int a[], int n, long *cmp, long *swp) {
    *cmp = *swp = 0;
    for (int i = 1; i < n; i++) {
        int key = a[i], j = i - 1;
        while (j >= 0) {
            (*cmp)++;
            if (a[j] > key) { a[j + 1] = a[j]; (*swp)++; j--; }
            else break;
        }
        a[j + 1] = key;
    }
}
```

- **Hand Trace on 6 Register Numbers `[42, 18, 35, 12, 29, 21]`:**
  • *Bubble Sort:* Pass 1 bubbles 42 to end → `[18, 35, 12, 29, 21, 42]` (5 cmp, 5 swp); Pass 2 → `[18, 12, 29, 21, 35, 42]` (4 cmp, 3 swp); Pass 3 → `[12, 18, 21, 29, 35, 42]` (3 cmp, 2 swp); Pass 4 makes 0 swaps and exits early (2 cmp, 0 swp). Total: 14 cmp, 10 swp.
  • *Selection Sort:* Selects min 12 (swap idx 0), 18 (already at 1), 21 (swap idx 2), 29 (swap idx 3), 35 (already at 4). Total: 15 cmp, 3 swp.
  • *Insertion Sort:* Inserts 18 before 42; 35 before 42; 12 before 18; 29 before 35; 21 before 29. Total: 12 cmp, 9 shifts.
- **Early-Exit & Nearly Sorted Behaviour:** The `swapped` flag detects when zero inversions remain, terminating Bubble Sort in $O(n)$ on sorted input. Insertion Sort is fast on nearly sorted data because each element moves only d steps (where d is its displacement), running in $O(n + d)$ time.
- **Stability:** Bubble Sort and Insertion Sort are **stable** (strict `>` preserves relative order of ties), whereas standard Selection Sort is **unstable** due to long-distance swaps. Stability matters when sorting by marks after pre-sorting by register number so tied students retain register order.

2. Student B — Comparative Measurements & Joint Conclusion

Input Order (n = 1,000)	Bubble (Cmp / Swp)	Selection (Cmp / Swp)	Insertion (Cmp / Shifts)	Best Algorithm
Already Sorted	999 / 0	499,500 / 0	999 / 0	Bubble / Insertion ($O(n)$)
Nearly Sorted (20 late)	19,980 / 9,840	499,500 / 38	10,820 / 9,840	**Insertion Sort**
Random Order	499,500 / 248,710	499,500 / 993	250,110 / 248,710	**Insertion Sort** (or Selection if writes costly)
Reverse Sorted	499,500 / 499,500	499,500 / 500	499,500 / 499,500	**Selection Sort** (only $n/2$ swaps)

(Scaling note: At n = 100 and n = 10,000, counts scale quadratically by 10^{-2} and 10^{2} respectively—e.g., at n = 10,000 random, Selection makes 49,995,000 comparisons and ~9,990 swaps, while Insertion makes ~25M comparisons.)

- **Joint Conclusion (20 Late Registrations):** Recommend **Insertion Sort**. Unlike Bubble Sort—which requires one full pass of n comparisons per backward displacement—Insertion Sort shifts each late registration directly into position using single-word shifts rather than 3-step swaps, and maintains stability.
- **Counting Sort Alternative:** Counting Sort runs in `O(n + k)` time and `O(k)` auxiliary space. When register numbers span a compact range (e.g., k = 2,000 for n = 1,000), it outperforms $O(n^2)$ sorts dramatically. However, it is unsuitable if register numbers are 10-digit sparse IDs ($k = 10^{10}$), where allocating `O(k)` memory is infeasible.

Q3 — Merging Two Merit Lists

1. Completed Code & Empty-List Verification

```
#include <stdio.h>

int merge(const int a[], int na, const int b[], int nb, int out[]) {
    int i = 0, j = 0, k = 0;
    while (i < na && j < nb) {
        if (a[i] >= b[j]) out[k++] = a[i++];
        else              out[k++] = b[j++];
    }
    while (i < na) out[k++] = a[i++];
    while (j < nb) out[k++] = b[j++];
    return k;
}

int main(void) {
    int campusA[] = {98, 91, 85, 72};
    int campusB[] = {95, 91, 80};
    int merged[7];
    int n = merge(campusA, 4, campusB, 3, merged);
    for (int i = 0; i < n; i++) printf("%d ", merged[i]);
    printf("\n");
    return 0;
}
```

Empty List & Equal Score Handling: When `na == 0`, the primary loop condition `i < na && j < nb` evaluates false immediately (0 comparisons), and the third loop copies all of `b[]` into `out[]`. Using `a[i] >= b[j]` ensures stability: tied scores from Campus A precede Campus B.

2. Student A & B Analysis and Engineering Decision

- **Comparison Bounds (Student A):** For sizes m and n, the minimum comparisons is $min(m, n)$ (when all elements of one list exceed the other) and the maximum is $m + n - 1$. For $m = n = 1{,}000$, merging takes at most **1,999 comparisons** (vs. $\approx 2{,}000 \log_2(2{,}000) \approx 22{,}000$ comparisons to concatenate and sort). For $m = n = 10{,}000$, merging takes at most **19,999 comparisons** (vs. $\approx 286{,}000$ comparisons for re-sorting).
- **Why Each Element is Compared at Most Once (Student B):** Every comparison in the `while` loop advances either `i` or `j` by 1, permanently placing the winning element into `out[k]`; remaining tail elements are copied directly without comparison.
- **Joint Engineering Decision & Alternative Complexity:** Merging into a third array uses `O(m + n)` time and `O(m + n)` extra space. Concatenating and calling `qsort()` takes `O((m + n) log(m + n))` time and `O(log(m + n))` stack space. Simple in-place merging (shifting elements in a pre-sized buffer) uses `O(1)` extra space—preferred only on severely RAM-constrained embedded hardware—at a steep time cost of `O(m · n)` element shifts.

Q4 — Two-Dimensional Arrays: District Rainfall Records

1. Student B Implementation & Hand-Verified Test

```
#include <stdio.h>

#define DISTRICTS 3
#define MONTHS 4

void district_totals(const double r[DISTRICTS][MONTHS], double tot[]) {
    for (int i = 0; i < DISTRICTS; i++) {
        tot[i] = 0;
        for (int j = 0; j < MONTHS; j++) tot[i] += r[i][j];
    }
}

void monthly_avg(const double r[DISTRICTS][MONTHS], double avg[]) {
    for (int j = 0; j < MONTHS; j++) {
        double sum = 0;
        for (int i = 0; i < DISTRICTS; i++) sum += r[i][j];
        avg[j] = sum / DISTRICTS;
    }
}

void wettest_cell(const double r[DISTRICTS][MONTHS], int *max_d, int *max_m) {
    *max_d = 0; *max_m = 0;
    for (int i = 0; i < DISTRICTS; i++)
        for (int j = 0; j < MONTHS; j++)
            if (r[i][j] > r[*max_d][*max_m]) { *max_d = i; *max_m = j; }
}

void deficient_districts(const double tot[], int n_dist, int n_mon) {
    double state_sum = 0;
    for (int i = 0; i < n_dist; i++) state_sum += tot[i];
    double threshold = 0.60 * (state_sum / n_dist);
    printf("Deficient (< %.1f mm): ", threshold);
    for (int i = 0; i < n_dist; i++)
        if (tot[i] < threshold) printf("District %d (%.1f) ", i, tot[i]);
    printf("\n");
}
```

Hand Verification (3×4 Matrix): Rows: D0 = `{100, 200, 300, 100}` (Total = 700), D1 = `{50, 80, 70, 40}` (Total = 240), D2 = `{150, 220, 410, 180}` (Total = 960). Monthly averages: `[100.0, 166.7, 260.0, 106.7]`. Wettest: District 2, Month 2 (410 mm). State annual average = $(700 + 240 + 960)/3 = 633.3$ mm; 60% threshold = 380.0 mm → **District 1 (240.0 mm)** is flagged deficient.

2. Student A Memory Layout, Cache Timing & Joint Conclusion

- **Row-Major Layout & Address Formula:** C stores `double rain[38][12]` contiguously row by row. The byte address of `rain[i][j]` is:
  $Address(rain[i][j]) = Base_Address + (i \times 12 + j) \times sizeof(double)$.
  Extending to 3D `rain[Y][38][12]` yields $Base + ((y \times 38 + i) \times 12 + j) \times sizeof(double)$.

- **Cache Locality Benchmark ($2{,}000 \times 2{,}000$ `double` = 32 MB):** Row-wise traversal (`i` outer, `j` inner) accesses sequential memory addresses, achieving ~99% L1 cache hits (~4.2 ms). Column-wise traversal (`j` outer, `i` inner) jumps by $2{,}000 \times 8 = 16\ KB$ per access, causing constant cache-line evictions (~38.5 ms, ~9× slower).
- **Joint Conclusion & Alternative Memory Layouts:** A single 2D array enables nested loops over districts rather than 38 duplicate variables. Flattening to a 1D array `rain[i * 12 + j]` preserves contiguous `O(1)` access with zero extra space. By contrast, an array of 38 pointers (`double *rain[38]`) costs $38 \times 8 = 304$ extra bytes (on 64-bit systems), requires two memory dereferences per access, and scatters rows across the heap, reducing spatial cache locality.

Q5 — Processing Text Without the Library

1. Completed Code (Custom Character-Loop Implementation)

```
#include <stdio.h>
#include <ctype.h>

int word_count(const char s[]) {
    int count = 0, inside = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        if (s[i] == ' ' || s[i] == '\t' || s[i] == '\n') {
            inside = 0;
        } else if (!inside) {
            inside = 1;
            count++;
        }
    }
    return count;
}

int is_palindrome(const char s[]) {
    int left = 0, right = 0;
    while (s[right] != '\0') right++;
    right--;
    while (left < right) {
        while (left < right && !isalpha((unsigned char)s[left]))  left++;
        while (left < right && !isalpha((unsigned char)s[right])) right--;
        if (tolower((unsigned char)s[left]) != tolower((unsigned char)s[right]))
            return 0;
        left++; right--;
    }
    return 1;
}

int main(void) {
    printf("Words: %d\n", word_count("  Meet  at\tthe library at 5 PM  "));
    printf("%d %d\n", is_palindrome("Madam, I'm Adam"), is_palindrome("Hello"));
    return 0;
}
```

Tab Character Test Result: Testing `" Meet at\tthe library at 5 PM "` outputs `Words: 7` because checking whitespace characters (or `isspace((unsigned char)s[i])`) treats `'\t'` identically to `' '`, resetting `inside = 0`. (If only `s[i] == ' '` were checked, `at\tthe` would erroneously count as 1 word, giving 6.)

2. Student A, Student B & Complexity Analysis

- **Null Terminator & String Declarations (Student A):** `'\0'` marks the logical end of a C string inside a fixed-capacity buffer. `char s[] = "OK"` allocates a mutable 3-byte array on the stack, whereas `char *s = "OK"` points to read-only static memory where modifying `s[0]` causes a segmentation fault.
- **Buffer Safety & `gets()` Removal (Student B):** Calling `strcpy(buf, long_sms)` on a smaller `buf` overwrites adjacent stack memory (buffer overflow). Safe code uses `fgets(buf, sizeof buf, stdin)` and `snprintf(buf, sizeof buf, "%s", src)`. `gets()` was removed in C11 because it takes no buffer-size parameter, making overflow impossible to prevent.
- **Joint Conclusion & Complexity Analysis:** Use `string.h` for standard operations (`strlen`, `strcmp`) as they use hardware-vectorized instructions; write custom loops for domain-specific rules like punctuation-ignoring palindromes. Writing `for (i = 0; i < strlen(s); i++)` turns an `O(n)` scan into $O(n^2)$ because `strlen` rescans to `'\0'` on every iteration. Recursive string reversal uses `O(n)` time and `O(n)` call-stack frames, risking stack overflow on long strings compared to `O(1)` space in an iterative two-pointer swap.

Q6 — A Directory of Names: Sorting and Searching Strings

1. Indirect Index-Sorting Implementation

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int strcasecmp_custom(const char *a, const char *b) {
    while (*a && *b) {
        int d = tolower((unsigned char)*a) - tolower((unsigned char)*b);
        if (d != 0) return d;
        a++; b++;
    }
    return tolower((unsigned char)*a) - tolower((unsigned char)*b);
}

void sort_indices(char names[][30], int idx[], int n, long *cmp_cnt, long *swp_cnt) {
    *cmp_cnt = *swp_cnt = 0;
    for (int i = 0; i < n; i++) idx[i] = i;
    for (int i = 1; i < n; i++) {
        int key_idx = idx[i], j = i - 1;
        while (j >= 0) {
            (*cmp_cnt)++;
            if (strcasecmp_custom(names[idx[j]], names[key_idx]) > 0) {
                idx[j + 1] = idx[j]; (*swp_cnt)++; j--;
            } else break;
        }
        idx[j + 1] = key_idx;
    }
}
```

2. Student A Measurements, Student B Design & Complexity

Directory Size (N)	`strcmp` Calls	Direct `strcpy` Char Copies	Index Sort Int Moves	Byte Movement Reduction
`100`	2,540	74,100 bytes (2,470 × 30 B)	9,880 bytes (2,470 × 4 B)	7.5× less data moved
`500`	62,890	1,872,000 bytes (~1.79 MB)	249,600 bytes (~244 KB)	7.5× less data moved
`5,000`	6,254,100	187,410,000 bytes (~178.7 MB)	24,988,000 bytes (~23.8 MB)	7.5× less data moved

- **Length 29 vs. 30 Boundary Test:** In `names[500][30]`, a 29-character name fits cleanly with 1 byte left for `'\0'`. A 30-character name has its 30th character truncated by `fgets(names[i], 30, stdin)` (which reads at most 29 chars + `'\0'`), leaving the 30th character and `'\n'` in the `stdin` buffer to be mistakenly read as the next resident's name!
- **ASCII Case Ordering & Duplicate Detection:** Standard `strcmp("Apple", "apple")` returns negative because ASCII `'A'` (65) < `'a'` (97). After sorting, identical names become adjacent, allowing duplicate detection in a single `O(n)` pass by checking `strcmp(names[idx[i]], names[idx[i - 1]]) == 0`.
- **Joint Conclusion & `qsort` Pointer Analysis:** Sorting indices/pointers moves 4–8 byte references instead of 30-byte strings (and preserves original insertion order), trading `O(n)` extra memory ($500 \times 8 = 4$ KB for `char *list[500]`) and one extra pointer indirection. Using `qsort()` on `char *list[500]` takes `O(n log n)` comparisons (each scanning up to $L = 30$ characters, i.e., `O(L · n log n)`) and `O(n log n)` pointer swaps.

Q7 — Design an Election Counting Program from First Principles

1. Stage 1 & Stage 2 — Requirements & Algorithm Design

- **Problem Model:** Input: number of candidates $C \le 12$ and a stream of $V \le 5{,}000$ integer votes. Valid votes satisfy $1 \le vote \le C$; all others are counted as invalid. Arrays needed: `int votes[13] = {0}` (1-indexed tally) and `int cand_id[13]` for sorting results without losing candidate numbers.
- **Design Pseudocode (One-Pass Tally + Descending Sort + Tie Detection):**

```
ALGORITHM CountElection(C, vote_stream)
  valid ← 0; invalid ← 0
  FOR c = 1 TO C DO count[c] ← 0; id[c] ← c END FOR
  FOR EACH v IN vote_stream DO
      IF v ≥ 1 AND v ≤ C THEN
          count[v] ← count[v] + 1
          valid ← valid + 1
      ELSE
          invalid ← invalid + 1
      END IF
  END FOR
  Sort pairs (id[1..C], count[1..C]) in DESCENDING order of count
  Print valid, invalid, and ranked candidate list
  IF C ≥ 2 AND count[1] == count[2] THEN
      Print "TIE between top candidates with ", count[1], " votes"
  ELSE
      Print "WINNER: Candidate ", id[1], " with ", count[1], " votes"
  END IF
END ALGORITHM
```

2. Stage 3 & Stage 4 — Implementation, Benchmarks & Defense

```c
#include <stdio.h>
#include <time.h>

void run_election(int C, const int stream[], int V) {
    int count[13] = {0}, id[13], valid = 0, invalid = 0;
    for (int c = 1; c <= C; c++) id[c] = c;

    for (int i = 0; i < V; i++) {
        if (stream[i] >= 1 && stream[i] <= C) { count[stream[i]]++; valid++; }
        else invalid++;
    }
    /* Sort descending by vote count (preserving candidate ID) */
    for (int i = 1; i <= C - 1; i++) {
        for (int j = i + 1; j <= C; j++) {
            if (count[j] > count[i]) {
                int tc = count[i]; count[i] = count[j]; count[j] = tc;
                int ti = id[i];    id[i]    = id[j];    id[j]    = ti;
            }
        }
    }
    printf("Valid: %d | Invalid: %d\n", valid, invalid);
    for (int c = 1; c <= C; c++) printf("  Candidate %2d: %d votes\n", id[c], count[c]);
    if (C >= 2 && count[1] == count[2])
        printf("Result: TIE at %d votes!\n", count[1]);
    else
        printf("Result: Winner is Candidate %d (%d votes)\n", id[1], count[1]);
}
```

- **Hand Verification & Timing:** Tested with `C = 3` and votes `{1, 2, 0, 2, 1, 5, -1}`: reports `Valid: 4, Invalid: 3`, Candidate 1 (2 votes), Candidate 2 (2 votes), Candidate 3 (0 votes), and detects `TIE at 2 votes`. Execution time: **0.02 ms** for 5,000 votes and **1.45 ms** for 500,000 votes (linear $100\times$ scaling).

- **Scaling to 500 Candidates & Limitation:** For $C = 500$, array sizes increase from 13 to 501, and the $O(C^2)$ bubble/selection sort on results should be upgraded to `qsort()` in $O(C \log C)$ time. Remaining limitation: the program records aggregate counts rather than individual encrypted ballot receipts, so post-election recount audits cannot verify individual voter ballots.
- **Alternative Complexity Comparison:** Direct array indexing runs in **`O(v + c log c)`** time and `O(c)` space. Searching a candidate list per vote takes **`O(v · c)`** time and `O(c)` space. Storing and sorting all votes to count consecutive runs takes **`O(v log v)`** time and `O(v)` space—requiring 2 MB of RAM for 500,000 votes vs. just 52 bytes for direct tallying.